Neural Wave AI Hackathon - Lifeware Challenge

The Neuron Migration: Use AI to Reverse-Engineer 30 Years of Insurance Logic



Lifeware Company Logo

1 About Lifeware

Founded in 1998 in Switzerland, Lifeware is a specialized software technology company that provides a fully integrated, web-based Software-as-a-Service (SaaS) platform for the insurance industry, especially for life insurance companies. Initially built by a team of mathematicians, physicists, and computer scientists, the company focuses on managing the entire policy lifecycle. Lifeware's core mission is to help life insurance companies migrate away from legacy infrastructure, providing software solutions that drive operational productivity, cost reduction, and greater market agility.

2 The Story

NeuralWave Life is a (fictional) life insurance company in Ticino. Since 1994 it has sold unit-linked policies, meaning that the customer's money is invested in investment funds. For more than 30 years a system called Neuron has calculated every premium, fee, fund unit and payout. Neuron started as a fork of an older system written in COBOL. Later, part of it was rewritten in Java as Neuron NG (New Generation). Since then it has been patched, its parameters have changed many times, and sometimes people corrected values by hand.

Unfortunately, Neuron is no longer actively maintained and has a declining developer base. Therefore NeuralWave Life has decided to migrate its systems to a third-party provider, Lifeware. Before the new system can go live it must calculate exactly the same values as Neuron, for every policy, to the cent. But there is a problem: nobody knows exactly how Neuron calculates these values. The documentation is incomplete, the original developers are gone, and there is no source code.

What you have is the data: a full export of the system, and the values Neuron calculated for a part of the policies. Your mission: find the rules and underlying formulas behind these values, and write them down as code that a legal auditor can check.

3 Why This Matters

Lifeware has this or similar problems in every real migration. Old systems contain decades of policy history, many product versions, parameter changes and silent bug fixes. We know the inputs (premiums, dates, ages, fund prices, parameters) and the outputs (commissions, statements, surrender values, fees). But we do not know the formulas in between.

A model that predicts a surrender value approximately does not help much. Insurance is strictly regulated: every amount paid to a customer must be exact, explainable and reproducible. You cannot tell an auditor that a

payout came from the hidden layers of a neural network. A value that doesn't match up completely is a case that a person has to investigate.

This is exactly where AI can make a big difference. Today, experts spend months comparing numbers, creating tools and sometimes guessing rules by hand. In this challenge, AI is the detective, not the calculator: machine learning, search and language models should help to find the rules and formulas, while the final result should be clear, deterministic code. A tool that does this well would save months of work in every migration.

4 The Data

The data is synthetic but realistic. It has the same structure and the same typical problems as real legacy exports:

- **23 related tables**: policies with their change history, insured persons, brokers, fund allocations, premiums, business events and about 8 million fund unit bookings. Market data contains fund prices, exchange rates and mortality tables.
- **Old-style formats**: short column names such as `PH_BEGDT`, dates stored as numbers like `1250131` (= 2025-01-31), fixed-point numbers and code lists. A data dictionary (`NRDICT`) and the code lists (`NRCODE`) are part of the export.
- **History**: 5,000 policies from 1994 to 2026, from the COBOL years, through the conversion, to many years of Neuron NG.
- **Manual corrections**: some values were corrected by hand over the years. Some of them are listed in the table `NREXCP`, but this list is (as usual) incomplete.
- **Targets**: the values Neuron calculated for 60 % of the policies (train set).

Another 20 % of the policies are public: their data is included, but the targets aren't, and they are scored on a live leaderboard. The last 20 % are private: you never see their data, and they are used as part of the final evaluation. A solution can therefore not memorize policies it has never seen, so it has to find the real rules and formulas.

The provided folder contains:

`legacy/`	the Neuron export on the extract date 2026-06-30, for the train and the public policies, plus all market and parameter data (`NR*.csv`)
`targets/`	the values Neuron calculated for the train policies: one file per task, plus diagnostic files for debugging
`SCHEMA.md`	a partial description of all tables, columns, keys, formats and target files

The target files are in a clean, modern format: comma-separated, ISO dates and decimal numbers. The `policy_id` is the policy number `PH_POLNR` without spaces.

Careful: use the booking table `NRBOOK` to understand and debug the accounting, not as a source for your formula. Once the new system goes live, there is no Neuron any more that writes bookings. The new system has to calculate the premium and charge bookings itself, from the premiums, the parameters and the market data.

5 The Challenge

There are five tasks, ordered from the easiest to the hardest, plus two bonus tasks. For each task you get the same four parts:

Goal: what exactly you have to calculate.

Why: why NeuralWave Life needs this value.

What we know: what the business analysts still remember. Like in a real migration, this is incomplete: some details are missing, simplified or out of date. The exact rules are hidden in the data, and finding them is the challenge.

Result: what your function returns, the name of the target file in `targets/`, and an example row of it.

You do not need to know anything about insurance: all terms used in the tasks are explained in the appendix (Key Terms).

ID	Task	What Neuron calculates
t0	Commission	The commission a broker earns on every premium the customer pays
t1	Next due dates	The days on which the premiums of the next year are due, and the days they are collected
t2	Projected maturity value	The value at the end of the policy, as projected in the offer the customer signed
t3	Surrender value	The full fund accounting of a policy, and the amount the customer gets on surrender
t4	Annual statement	The yearly overview of the premiums and all costs, sent to every customer
Bonus Tasks		
t5	Outliers	Find the values that were changed by hand and follow no rule
t6	Visualization/Tools	Create visualizations that help to find and explain the rules

Some tasks build on each other: t4 uses the charges of t3. On the way you will meet the traps that every migration team knows: parameters that are valid on the application date and not today, policy data with a history, and a system change. The COBOL and Java versions of Neuron do not calculate everything in the same way.

t0 Commission (warm-up)

Goal: calculate the commission for every premium item that was received and not reversed later.

Why: NeuralWave Life does not sell its policies directly; brokers do. A wrong commission means either an angry broker or lost money.

What we know:

- The commission is a percentage of the received premium amount (`PR_AMTRC`).
- The rate is in `NRCOMR`. It depends on the broker tier and on the type of premium: `E` for single-premium policies, `L` for regular-premium policies.
- A policy can change its broker over the years.

Result: {`(policy_id, due_nr): amount`}, file `t0_commission.csv`.
Example: `BL10000013,1,9.45` is the commission of premium item 1 of policy `BL10000013`.

t1 Next due dates

Goal: for every regular-premium policy in state `10` on the extract date 2026-06-30, calculate all premium due dates of the following year (after 2026-06-30, up to and including 2027-06-30), each with its collection date.

Why: with a direct debit, NeuralWave Life takes the money from the customer's account itself. If a date is wrong, the money is taken on the wrong day, and the customer complains. The finance department also needs these dates to plan how much money comes in on which day.

What we know:

- The due dates follow the premium frequency from the start date. There is no due date on or after the maturity date.
- Neuron maintains the schedule itself. The past due dates in `NRPREM.PR_DUEDT` show how it behaves, so study them carefully.
- Assume that nothing changes on the policy after the extract date.

Result: {`(policy_id, n): (due_date, collection_date)`}, where n = 1, 2, ... numbers the due dates after the extract date. File `t1_next_due_dates.csv`.
Example: `BL10000013,1,2026-07-01,2026-07-01` is the first due date of policy `BL10000013` after the extract date.

t2 Projected maturity value

Goal: calculate the value at maturity that the offer projected for every policy.

Why: before signing, the customer receives an offer that shows what the policy could be worth at maturity if the funds grow at an assumed rate. The law requires this projection, and customers keep their offer: if they complain years later, the insurer must be able to show exactly how the number was calculated.

What we know:

- The projection simulates the policy month by month, from the start date to the maturity date, as if the funds grew every month at the assumed yearly rate of `NRILLU`.
- It is a simplified accounting: there are no real fund prices, no currencies and no individual agreements, and the whole value is treated as one amount.
- The diagnostic file shows the projected value at the end of every policy year. Use it to find out where your projection starts to differ.

Result: {`policy_id: maturity_value`}, file `t2_projected_maturity_value.csv`. The diagnostic file `t2_projection_year.csv` contains the projected value at the end of every policy year.
Example: `BL10000013,109748.88`.

t3 Surrender value

Goal: calculate the surrender value of every policy on the given valuation dates.

Why: the surrender value is the most important number in the system. It is what a customer gets if they end the policy, and it is the basis of the company's balance sheet.

What we know:

- Charges cancel units: On every monthly anniversary, Neuron takes the admin fee, the management charge (with a different rate per pot) and the risk charge by cancelling units in all funds and pots. These bookings exist in `NRBOOK`, but their amounts and units are missing, so you have to calculate them.
- Customer orders are complete: Fund switches, withdrawals and surrenders are orders of the customer, and their bookings are complete in the export. Reversed events and technical corrections can be ignored.
- Surrender value: Value the units at the redemption price (the last price on or before the valuation date) in the policy currency.
- Surrendered policies and policies of insured persons who died are worth 0.

Result: {`(policy_id, valuation_date): surrender_value`}, file `t3_surrender_value.csv`. The train targets use the valuation dates 2006-12-31, 2020-04-30 and 2025-12-31. Three diagnostic files help to check each step: `t3_booking.csv` (amounts and units of the missing bookings), `t3_holding.csv` (units per fund and pot) and the column `policy_value` (value before the penalty).
Example: `BL10000013,2025-12-31,31111.18,31111.18`.

t4 Annual statement

Goal: calculate the annual statement of every policy and calendar year, from the start year to the year the policy ends by surrender or death, at the latest 2025.

Why: every year, each customer receives a statement of what they paid and what the policy cost them. The law requires this cost transparency, and the numbers must be consistent with the fund accounting.

What we know: a statement has six columns. The charges are exactly the ones that Neuron takes from the policy in the fund accounting of t3, so t3 and t4 are best solved together.

`premiums`	premiums received in the year (money date)
`entry_charges`	entry charges on the premiums invested in the year (trade date)
`admin_fees`	admin fees deducted in the year
`management_fees`	management charges deducted in the year
`risk_charges`	risk charges deducted in the year
`total_charges`	entry + admin + management + risk

Result: {`(policy_id, year): (premiums, entry_charges, admin_fees, management_fees, risk_charges, total_charges)`}, file `t4_annual_statement.csv`.
Example: `BL10000013,2021,2835.00,103.95,78.00,187.88,69.64,439.47`.

t5 Outliers (bonus)

Goal: find the values in all previous tasks that were overwritten by hand.

Why: manual corrections follow no rule, so the new system cannot calculate them. A migration team must find them and take them over one by one.

What we know:

- some are listed in `NREXCP`, but not all.
- some tasks have outliers, while some might not.
- unlike the other tasks, t5 gets the target files of the policies it has to check, in the same format as `targets/`. A value that differs from what your rules calculate is either a manual correction or a rule you have not found yet.

Result: a set of `(task, key)`, for example `("t2_projected_maturity_value", "BL10000042")` or `("t4_annual_statement", ("BL10000042", 2021))`. It is compared with all known manual corrections, listed or not, using the F1 score.

t6 Visualization/Tools (bonus)

Goal: show the visualizations and tools you used to find and explain the rules.

Why: a good visualization makes a hidden rule visible at a glance, for a person or as input for another model. In a real migration, it is also how the team convinces the client that the rules are right.

Ideas:

- the differences between your values and the targets, over the application date or the processing date
- the differences over the number of days to the next birthday
- a calendar heat map of due dates and collection dates
- a waterfall chart of an annual statement
- the timeline of one policy: premiums, charges, units and value
- the distribution of cent differences per task

6 Where AI Comes In

The tasks are too many and too detailed to solve by hand in one weekend. The teams that do best will let AI do most of the work. Classic machine learning, search methods and language models all have their place:

- **Find where a rule changes:** train a decision tree or a rule learner to predict which rows your current rule gets wrong. Its splits point to hidden conditions, for example "application date before a certain day" or "product family X". Change-point detection on the differences over time finds the dates when the system behaved differently.
- **Find the formula:** linear or robust regression recovers rates and factors from inputs and outputs. Symbolic regression and program synthesis search for the expression that matches the targets exactly, including rounding and date rules.
- **Find the structure:** clustering groups policies that behave in the same way, and feature importance shows which columns matter for a target. Both help to decide where to look first.
- **Find the noise:** anomaly detection (for example an isolation forest on the remaining differences) separates the manual corrections from rules you have not found yet (bonus task t5).
- **Build the tools with LLMs:** language models are very good at writing the code around the search: data loaders for the old formats, policy timeline viewers, dashboards of differences and other visualizations. Such tools make the data easier to understand, for a person, for a machine learning model or for another LLM. An LLM can also turn the conditions found by a decision tree into readable code, or propose rules from a task description.

The strongest solutions will combine these methods into one approach that works for all tasks, and for new targets too, instead of solving each task in a different way.

7 Submission

A submission is a Python package with one entry point, `submission.py`, which defines one function per task. Each function gets the path `legacy_dir` of a folder with the `NR*.csv` files and returns the calculated values. Dates are `datetime.date`, amounts are `decimal.Decimal`. A missing function scores 0 for its task.

```
def t0_commission(legacy_dir) -> dict[tuple[str, int], Decimal]

def t1_next_due_dates(legacy_dir, extract_date) -> dict[tuple[str, int], tuple[date, date]]

def t2_projected_maturity_value(legacy_dir) -> dict[str, Decimal]

def t3_surrender_value(legacy_dir, valuation_dates) -> dict[tuple[str, date], Decimal]

def t4_annual_statement(legacy_dir) -> dict[tuple[str, int], tuple[Decimal, ...]]

def t5_outlier_detection(legacy_dir, targets_dir) -> set[tuple[str, object]]
```

- **Python only, any library:** you can use any external library. List them in a `requirements.txt` file. The evaluator installs them and runs your code offline on the private data, within a fixed time limit.
- **Rules, not results:** your code must calculate every value from the data it receives. It must not read the target files or contain target values. The only exception is t5, which gets the folder `targets_dir` with the target files of the policies to check. The private policies are unseen, so memorizing does not help anyway.
- **Show your work:** add the tools and visualizations you built, and a short `README.md` that explains your approach and the rules you found.

How you find the rules is completely up to you. The Python code is only the output format of the underlying formula, and it can be produced in many ways: generated from the conditions of a decision tree or a formula found by symbolic regression, written by an LLM from such a formal input, or written by hand using what your models found in the data. We ask for Python because a formula written as code is the most general format: every approach, from pure machine learning to hand-written rules, can produce it, and every result can be run and checked in the same way.

You can test your solution locally with the files in `targets/`. They have the same format as the evaluation data.

8 Evaluation

In insurance "almost right" is not enough, so a value counts only if it is exact: an amount must be right to the cent, and a date right to the day. A row of the annual statement counts only if all six columns match. Outlier values are not scored. Solutions are evaluated in these ways:

- **Accuracy:** the share of exactly correct rows of each task, from 0 to 1, on the public and private policies. Every correct row counts, so even a simple first version earns points, and every rule you find adds more. A task where all rows are correct earns extra points as a "perfect match" bonus.
- **Maintainability:** the new system must be maintained for many years, so the code should be easy to read and change. We measure the maintainability index of your Python code with the radon library. It combines the size of the code (Halstead volume and lines of code), its cyclomatic complexity and its share of comments into one number between 0 and 100, where higher is better. Short, simple functions with a generic formula, few branches and useful comments raise the index. Only the source files for the tasks will be taken into account. This means for the tools/visualizations your are not even restricted to python.
- **Bonus tasks:** points for the outliers of t5 (F1 score against all known manual corrections on the public and private policies) and for the visualizations of t6.
- **Generalization:** points for approaches that can be applied to new problems. If the client asks for the formula of another target tomorrow, how much work would it be to apply your approach to it? Approaches that find rules automatically, with tools that are not built for a single task, score best here.
- **Live Leaderboard:** a public leaderboard shows the accuracy of each task on the public policies and the maintainability index of each submission, so teams can follow their progress during the hackathon.

9 Proposed Approach

There is no single correct method, but here is one way that works. Our own reference solution gets 100 % on every task with fewer than 300 lines of Python. Most steps can be done or supported by AI.

1. **Understand the data:** load the tables, decode the old date formats and code lists, and follow one policy from application to surrender: premiums, events, bookings. A policy timeline viewer, for example written with the help of an LLM, is worth building early.
2. **Start with the story:** turn the business description of a task into a first, simple implementation. The result is usually correct for most rows.
3. **Look at the mistakes:** compare with the train targets and study the rows that do not match. Plot the differences against the application date, the processing date, the cents of the amount or the days to the next birthday. Let a decision tree or change-point detection find the pattern. Sudden jumps point to system changes, and repeating patterns point to hidden conventions such as rounding, day counting or holidays.
4. **Test ideas automatically:** turn each pattern into a few possible rules, and test each rule for exact matches on the train data. A symbolic search can try all combinations of rounding modes, date shifts and parameter lookups, regression can recover unknown factors, and an LLM can suggest new candidate rules.

5. **Use the diagnostics:** for the hard tasks, the intermediate targets (projected values per year, fund units, single charge bookings) let you check each step of the calculation, not only the final number.
6. **Separate rules from noise:** when the rules explain almost everything, what is left is either a rule you have not found yet, or a value changed by hand. Anomaly detection helps to tell them apart, which is the bonus task t5.

A good solution is more than a high score. It is clear code that a developer can maintain and an auditor can check. The best teams will build an AI-driven method that can find such rules for any legacy system, not only the answers for this one.

10 Additional Information

You do not need any knowledge of insurance or finance. The task descriptions, the Key Terms in the appendix and `SCHEMA.md` explain probably everything you need. In this data, curiosity and good tools matter more than domain knowledge.

A Key Terms

These are all the terms used in the tasks. On the right you find the tables or columns where the term appears in the data; `SCHEMA.md` has the details.

Policy and people

Policy	the insurance contract between NeuralWave Life and a customer. Every change to a policy creates a new *version*, so the table shows what was valid on which day.	NRPOLH
Insured person	the person whose life is insured, with birth date, sex and smoker status.	NRRISK, NRPERS
Broker	an independent sales agent who sold the policy. A policy can change its broker over the years.	NRBROK
Broker tier	the category of a broker: `A`, `B` or `C`. Commission and entry charge rates depend on it.	BR_TIER
Product, tariff	the kind of policy that was sold, and its price list of charges and rates. Prices change over time, so every row has a validity period.	NRPROD, NRCHRG
Individual agreement	a special rate negotiated for one policy, which replaces the normal rate.	NRAGRE
Policy state	`10` in force and paying premiums, `20` paid-up (no more premiums), `30` premium holiday, `60` surrendered, `70` death.	PH_STATE

Dates

Application date	the day the customer signed the offer.	PH_APPDT
Start date, maturity date	the first and the last day of the contract.	PH_BEGDT, PH_ENDDT
Policy year	a year of the contract, starting on the anniversary of the start date.	
Monthly anniversary	the same day in every month as the start date: a policy that starts on 15 March has its monthly anniversaries on the 15th.	
Money date	the day a payment of the customer arrived.	PR_RCVDT, BK_RCVDT
Trade date	the day fund units were actually bought or sold.	BK_TRDDT

Business day a working day in the local calendar.

Extract date the day the export was taken: 2026-06-30.

Valuation date the day for which a value is calculated.

Money in

Premium the money the customer pays. A *single premium* is paid once at the beginning. *Regular premiums* are paid again and again, for example monthly or yearly: this is the *frequency*. PH_FREQ

Premium item one expected premium payment, with its due date, and the amount and date it was received. NRPREM

Due date the day a premium must be paid. PR_DUEDT

Collection date the day the money is actually taken from the customer's account.

Direct debit NeuralWave Life takes the money from the customer's bank account itself (`LS`). PH_PAYMT

Bank transfer the customer sends the money (`UB`). PH_PAYMT

Funds

Unit-linked policy a policy that works like a small investment account: the premiums buy units of investment funds, and the value of the policy is the value of these units.

Fund, unit an investment fund is split into units. The value of a policy is the number of units it holds times the unit price. NRFUND

Issue price the price for buying a unit. It is higher than the redemption price. NRFPRC

Redemption price the price for selling a unit. NRFPRC

FX rate the exchange rate between currencies. A fund can be in another currency than the policy. NRFXRT

Fund allocation how a premium is split between funds, in percent. NRALLO

Pot units are kept in two pots: *initial units* (pot 1), bought with the premiums of the first months, and *base units* (pot 2). They have different management charges. BK_POTCD

Booking one movement of units: bought, sold or cancelled. NRBOOK

Charges

Entry charge a percentage of every premium that NeuralWave Life keeps before the rest is invested. NRENTC

Admin fee a fixed amount per month for running the policy. CG_ADMFEE

Management charge a yearly percentage of the fund value for managing the investments, taken every month. CG_AMCP1, CG_AMCP2

Death benefit the amount NeuralWave Life pays if the insured person dies.

Sum at risk the part of the death benefit that is more than the value of the units: the amount the insurer really risks.

Death probability — the probability that a person of a given age and sex dies within one year, from a mortality table. NRMORT

Risk charge — the monthly price for the death benefit. It depends on the sum at risk and on the death probability. A *loading* is an extra safety margin, and smokers pay more with a *smoker factor*. CG_SMKFAC

Money out and events

Commission — the money a broker earns for selling a policy, paid on every premium the customer pays. NRCOMR

Withdrawal — the customer takes out a part of the value, and the policy continues.

Surrender — the customer ends the policy early and gets its value paid out, minus a *surrender penalty*. NRSURR

Waiver — a period in which no surrender penalty is charged. NRWAIV

Business event — anything that happens to a policy: issue, premium receipt, fund switch, withdrawal, surrender, death and more. NREVNT, NRCODE

Reversal — an event (`900`) that cancels an earlier event, for example a premium that came back from the bank.

Technical correction — an internal fix of the system (event `990`).

Manual correction — a value that a person overwrote by hand. Some are listed in `NREXCP`, but not all. NREXCP